

```

# =====
# Data Science Assignment
# =====

# SECTION 1: Setup
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# For visualization aesthetics
sns.set(style="whitegrid", palette="muted", font_scale=1.2)

# Google Drive URLs (update if needed)
trader_data_url = "https://drive.google.com/uc?id=1IAfLZwu6rJzyWKgBToqwSmmVYU6VbjVs"
sentiment_data_url = "https://drive.google.com/uc?id=1PgQC0t08XN-wqkNyghWc_-mnrYv_nhS"

# SECTION 2: Load Data
trades = pd.read_csv(trader_data_url)
sentiment = pd.read_csv(sentiment_data_url)

print("Trader data shape:", trades.shape)
print("Sentiment data shape:", sentiment.shape)
print(trades.head())

# SECTION 3: Clean and Preprocess
trades['time'] = pd.to_datetime(trades['time'], errors='coerce')
sentiment['Date'] = pd.to_datetime(sentiment['Date'], errors='coerce')

# Drop missing or invalid entries
trades.dropna(subset=['closedPnL', 'leverage', 'time'], inplace=True)

# SECTION 4: Feature Engineering
# Extract daily metrics
trades['date'] = trades['time'].dt.date
daily_summary = trades.groupby('date').agg({
    'closedPnL': 'mean',
    'size': 'sum',
    'leverage': 'mean'
}).reset_index()

# Merge with sentiment data
merged_df = daily_summary.merge(sentiment, left_on='date', right_on='Date', how='inner')

# SECTION 5: Exploratory Data Analysis
plt.figure(figsize=(8,5))
sns.boxplot(x='Classification', y='leverage', data=merged_df)
plt.title("Leverage Distribution: Fear vs Greed")
plt.savefig('outputs/leverage_boxplot.png')
plt.show()

# Correlation analysis
correlation = merged_df[['closedPnL', 'size', 'leverage']].corr()
sns.heatmap(correlation, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.savefig('outputs/correlation_matrix.png')

```

```
plt.savefig('outputs/correlation_matrix.png')
plt.show()

# SECTION 6: Modeling (Example)
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder

merged_df['is_greed'] = LabelEncoder().fit_transform(merged_df['Classification'])
X = merged_df[['leverage', 'size', 'closedPnL']]
y = merged_df['is_greed']

model = LogisticRegression()
model.fit(X, y)

print("Model coefficients:", model.coef_)
print("Intercept:", model.intercept_)

# Save processed data
merged_df.to_csv("csv_files/merged_summary.csv", index=False)
```

Trader data shape: (211224, 16)

Sentiment data shape: (2644, 4)

	Account	Coin	Execution Price	\
0	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9769	
1	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9800	
2	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9855	
3	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9874	
4	0xae5eacaf9c6b9111fd53034a602c192a04e082ed	@107	7.9894	

	Size Tokens	Size USD	Side	Timestamp IST	Start Position	Direction	\
0	986.87	7872.16	BUY	02-12-2024 22:50	0.000000	Buy	
1	16.00	127.68	BUY	02-12-2024 22:50	986.524596	Buy	
2	144.09	1150.63	BUY	02-12-2024 22:50	1002.518996	Buy	
3	142.98	1142.04	BUY	02-12-2024 22:50	1146.558564	Buy	
4	8.73	69.75	BUY	02-12-2024 22:50	1289.488521	Buy	

	Closed PnL	Transaction Hash	Order ID	\
0	0.0	0xec09451986a1874e3a980418412fcd0201f500c95bac...	52017706630	
1	0.0	0xec09451986a1874e3a980418412fcd0201f500c95bac...	52017706630	
2	0.0	0xec09451986a1874e3a980418412fcd0201f500c95bac...	52017706630	
3	0.0	0xec09451986a1874e3a980418412fcd0201f500c95bac...	52017706630	
4	0.0	0xec09451986a1874e3a980418412fcd0201f500c95bac...	52017706630	

Next steps: [Explain error](#)

	Crossed	Fee	Trade ID	Timestamp
0	True	0.345404	8.950000e+14	1.730000e+12
1	True	0.005600	4.430000e+14	1.730000e+12
2	True	0.050431	6.600000e+14	1.730000e+12
3	True	0.050043	1.080000e+15	1.730000e+12
4	True	0.003055	1.050000e+15	1.730000e+12

KeyError Traceback (most recent call last)
 /usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)

```

3804         try:
-> 3805             return self._engine.get_loc(casted_key)
3806         except KeyError as err:

```

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in
 pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in
 pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'time'

The above exception was the direct cause of the following exception: